

Dr. Mundo

Agentic AI Cost Estimator for Philippine Healthcare

1. Overview

Dr. Mundo answers plain-language or Taglish questions about what medical procedures and outpatient services cost in the Philippines. It is an **agentic** system: a language model (gpt-4o-mini) decides which tools to call, retrieves real figures from a local SQLite database, and returns a grounded estimate. It never invents numbers and never gives medical advice.

It routes every question down one of two paths:

- **Path A - Covered procedures** (a PhilHealth case rate applies): reports the hospital price range, the PhilHealth case rate, and the estimated out-of-pocket (OOP).
- **Path B - Outpatient services** (not PhilHealth-covered): reports the price range only and states clearly that it is not covered (never computes OOP).

2. How it works

Request flow: **Streamlit UI** -> **FastAPI** (POST /ask) -> **DrMundoService** -> **Input guard** (PII redaction + topic classification) -> **ReAct tool-calling loop** -> tools -> **Output guard** -> response. Each request is logged to MLflow.

Tools available to the agent:

- **search_catalog** - semantic search (embeddings + a curated Taglish alias boost) over the procedure / service catalog.
- **get_covered_cost** - parameterized SQL for Path A (case rate, price range, OOP).
- **get_outpatient_cost** - parameterized SQL for Path B (price range).
- **list_hospitals** - list or resolve a hospital mention.
- **ask_user** - ask one clarifying question when the request is ambiguous.

Why numbers cannot be hallucinated:

The structured answer fields are filled from the tool observations, not from the model's prose. The output guard then cross-checks every peso figure in the prose against the set of retrieved values; if any figure is unmatched, it discards the prose and re-renders the answer deterministically from the grounded fields.

3. Guardrails and hard constraints

- **No text-to-SQL** - the model never writes SQL; it only picks which parameterized query to call.
- **No invented numbers** - every figure comes from a database row (enforced twice).
- **No HMO / private insurance** and **no PhilHealth policy / eligibility** questions.
- **No document vector store** - embeddings match the catalog only.
- **No live external lookups** at answer time - all data is local CSV / SQLite.
- **No medical advice** - clinical / diagnostic / treatment questions are politely refused.

4. Data model

Four tables in the local SQLite database (dr_mundo.db):

Table	Rows	Purpose
hospitals	5	id, hospital, city
philhealth_procedure_rates	4,312	full Annex B catalog: rvs_code, procedure, case_rate
hospital_procedure_prices	34	Path A prices; only 10 covered procedures are actually priced
hospital_prices	100	Path B: 47 distinct outpatient services (CT, MRI, labs, checkups)

Out-of-pocket rule (Path A): case rate $\geq$ price high $\rightarrow$ may be fully covered (OOP 0); case rate $\geq$ price low $\rightarrow$ OOP 0 up to (high - case rate); otherwise OOP is (price - case rate) across the range.

5. Monitoring and evaluation (Phase 9)

- **MLflow monitoring** - one run per request logging latency, token usage and estimated cost, the tool-call trace, prompt version, and grounding / error status. Implemented as a transparent wrapper around the OpenAI client, so it never changes an answer.
- **Eval harness** - 35 database-grounded cases across covered / outpatient / ambiguous / out-of-scope, scored for catalog match, path routing, coverage-OOP correctness, refusal and clarification correctness, latency, and token cost, with a prompt-variant ablation.

Prompt-variant ablation results (35 cases each):

Prompt	Pass	Routing	Match	Cov/OOP	Refusal	Clarify	Cost (USD)
system_v1 (baseline)	91%	100%	100%	93%	100%	50%	0.0224
system_v2 (concise)	94%	100%	100%	100%	100%	50%	0.0206
system_v3 (clarify-first)	94%	92%	80%	100%	100%	100%	0.0196

Takeaway: v3's aggressive clarify-first behaviour fixes all ambiguous cases but over-clarifies on legitimate outpatient queries (routing/match drop). **system_v2** is the best balance - top accuracy everywhere except the deliberately vague cases, at the lowest-but-one cost.

6. Technology stack

Layer	Technology
Language / runtime	Python 3.11
LLM + embeddings	OpenAI gpt-4o-mini, text-embedding-3-small
API	FastAPI + Uvicorn
UI	Streamlit
Data / retrieval	SQLite, NumPy (embeddings), Pydantic v2
Monitoring / eval	MLflow
Packaging	Docker + docker-compose (multi-stage image)

7. Repository layout

Path	Responsibility
config.py	paths, model names, usage-tracked OpenAI client
data/	CSV $\rightarrow$ SQLite build (load_db.py); committed catalog embeddings
db/	parameterized SQL queries, hybrid catalog search, Taglish aliases

Path	Responsibility
agent/	schemas, tools, ReAct loop, memory, formatter, versioned prompts, service
guardrails/	PII redaction, input topic classifier, output grounding guard
api/	FastAPI app: /ask, /health, /reset, /docs
ui/	Streamlit chat client
monitoring/	token/cost accounting + MLflow logging (Phase 9a)
eval/	offline eval harness + ablation (Phase 9b)
tests/	unit tests (aliases, queries, guardrails, monitoring, eval)

Repository: <https://github.com/Dawsxn/DrMundo> · Estimates only - not medical or financial advice.